Algorithmic Translation of Unquantized Audio Transient Indices into Musical Tempo Maps

1. Introduction to Digital Rhythm Analysis and Metrical Grid Extraction

The computational translation of an unquantized sequence of human-generated transient indices into a highly structured musical tempo map represents a foundational challenge within the discipline of Music Information Retrieval (MIR). In the specific context of a dependency-free C++ engine compiled to WebAssembly (WASM), the system is provided with a strictly one-dimensional array of audio sample indices. These indices represent the precise onset times of beatboxing transients extracted during a prior signal processing stage. Unlike sequenced or strictly metronomic rhythms, human beatboxing introduces continuous micro-timing fluctuations, dynamic and intentional tempo drift such as ritardandos and accelerandos, and high degrees of syncopation where strong metrical beats may be entirely devoid of acoustic energy.[1]

The primary objective of the algorithm is to mathematically interpret the spatial relationships between these raw transient hit points to extract a metrical-domain grid, translating a time-domain vector into a structured sequence of bars, beats, and subdivisions. This requires an exhaustive algorithmic pipeline that executes deterministically, relying on mathematical formalisms rather than heuristic guesswork or external libraries. The analysis must proceed through two critical and interconnected analytical phases. The initial phase involves identifying the underlying global pulse using an all-order Inter-Onset Interval (IOI) histogram clustering approach.[3] The secondary phase deploys a multi-agent beat tracking architecture to navigate local tempo variations, predict subsequent beat locations, and dynamically adjust to human rhythmic drift.[3]

Furthermore, the mathematical models must be sufficiently robust to dynamically incorporate user-defined hard constraints, referred to as anchors, which force the localized recalculation of the grid.[4] The system must also mathematically support complex fractional rhythm domains, including mixed meters and tuplets, without destroying the continuity of the surrounding global tempo map.[5] The following report details the complete algorithmic logic, mathematical formalizations, and structural C++ designs required to engineer this transient-to-MIDI translation engine.

2. Foundational Rhythmic Analysis: All-Order

Inter-Onset Interval Histograms

To deduce the global tempo and the underlying subdivision grid from an array of sample indices, the system must first identify the dominant periodicities hidden within the audio signal. Because human performance is inherently flawed and highly syncopated, algorithms that rely solely on the temporal distance between adjacent transients routinely fail. A beatboxer frequently utilizes syncopation, placing acoustic energy on weak subdivisions while leaving the primary downbeat completely silent. The mathematical solution to this missing pulse phenomenon is the computation of an all-order Inter-Onset Interval (IOI) analysis.[3]

2.1. Mathematical Formulation of All-Order Inter-Onset Intervals

An Inter-Onset Interval is traditionally defined as the temporal difference between two successive acoustic events. However, to capture the overarching metrical structure of a syncopated performance, the system must extend this definition to compute the time intervals between all pairs of events, including those separated by intervening note onsets.[3]

Let the input array of transient sample indices be defined as $T = [t_1, t_2, \ldots, t_N]$, where the indices are monotonically increasing. Let F_s be the sample rate of the audio file, which is typically 44100 Hz.[3] To bound the computational complexity of the algorithm, a maximum perceptual limit for a musical interval must be established. Research indicates that the human perception of a musical pulse generally deteriorates for intervals longer than 2.5 seconds.[3] Therefore, the maximum sample distance S_{max} is defined as $2.5 \times F_s$ samples.

The set of all valid inter-onset intervals I is constructed via a bounded combinatorial operation. The engine iterates through the array T, calculating the difference between a transient t_i and subsequent transients t_j, terminating the inner loop when the difference exceeds S_{max}. The mathematical definition of the set I is:

$$I = \{(t_j - t_i) \mid \forall i, j \text{ such that } j > i \text{ and } (t_j - t_i) \leq S_{max}\}$$

By calculating intervals between non-adjacent hits, the system mathematically compensates for the "missing pulse".[1] For example, if a beatboxer performs a pattern where the downbeat is silent, the temporal distance between the preceding upbeat and the following upbeat will still equal an exact integer multiple of the primary beat. The all-order IOI calculation ensures that these latent structural intervals are captured and represented within the dataset, allowing the algorithm to deduce the existence of a grid line even when no direct acoustic event triggered

it.[3]

2.2. Constructing the Continuous Histogram and Applying Gaussian Smoothing

The raw intervals collected in set I represent continuous human timing. Consequently, identical musical intervals will not produce identical sample distances; they will instead be distributed around a statistical mean. To extract the true periodicities, the continuous values in I must be aggregated into a discrete histogram and subsequently smoothed.[3]

In a dependency-free C++ environment, maintaining a histogram at a one-sample resolution is both memory-intensive and analytically counterproductive due to signal sparsity. Instead, the algorithm must define a discrete histogram H with a reduced bin resolution Δb. A resolution of approximately 10 to 11.6 milliseconds is standard in MIR tempo induction algorithms, which equates to roughly 441 to 512 samples at a 44.1 kHz sample rate.[7]

For every interval $x \in I$, the algorithm increments the corresponding bin H. Because human micro-timing variations cause intervals to spread across adjacent bins, the raw histogram H will present noisy, jagged peaks. To resolve this, the system applies a Gaussian smoothing kernel to the histogram. Let $G(\sigma)$ be a Gaussian kernel with a standard deviation σ corresponding to the expected variance in human timing (e.g., 20-30 milliseconds). The smoothed histogram H_{smooth} is computed through a discrete convolution:

$$H_{smooth}[k] = \sum_{m=-M}^{M} H[k-m] \cdot e^{-\frac{m^2}{2\sigma^2}}$$

Following the convolution, the algorithm isolates the local maxima within H_{smooth}. A specific bin $H_{smooth}[k]$ is classified as a peak if its magnitude is strictly greater than its neighbors within a predefined local window (typically a 5-point to 9-point running window).[10] The collection of these local maxima forms the set of cluster centroids $C = [c_1, c_2, \ldots, c_P]$, representing the most prominent rhythmic periodicities present in the audio signal, measured in samples.

Phase	Algorithmic Component	Mathematical Operation	Implementation Rationale
1	Bounded Pairwise Subtraction	$(t_j - t_i) \leq S_{max}$	Prevents $O(N^2)$ explosion; restricts intervals to the perceptual limit of 2.5 seconds.
2	Discrete Binning	$H \mathrel{+}= 1$	Quantizes raw sample distances into localized density representations.
3	Gaussian Convolution	$H * G(\sigma)$	Smooths micro-timing fluctuations, merging adjacent bins into unified continuous peaks.
4	Local Maxima Extraction	$H_{smooth}[k] > \max($	Isolates the specific periodicities that act as candidate pulse hypotheses.

2.3. Extracting the Dominant Pulse via the Two-Way Mismatch Error Function

The extracted peaks in set C represent multiple overlapping metrical levels. A peak might represent a 16th-note subdivision, a quarter-note tactus, or a full measure. The objective is to mathematically isolate the fundamental tactus (the primary beat) and the tatum (the smallest rhythmic subdivision) from these competing hypotheses.[10]

Because rhythmic periodicities in Western music are related by small integer ratios, a valid tempo hypothesis will be supported by the presence of its harmonics and sub-harmonics.[3] A peak representing a quarter note will inherently be accompanied by peaks representing the eighth note (a 1:2 ratio) and the half note (a 2:1 ratio). To formalize this, the engine employs a scoring algorithm that evaluates the harmonic support of each candidate peak. One highly effective mathematical approach is the Two-Way Mismatch (TWM) error function, adapted for periodicity detection.[10]

For each candidate period $c_i \in C$, the algorithm generates a theoretical pulse train based on that period. The TWM function then calculates an error score by evaluating two distinct conditions. The first condition, known as positive evidence, measures how well the empirical peaks in the IOI histogram explain the theoretical pulse. The algorithm searches for the closest histogram peak to each harmonic of the candidate period and calculates the deviation. The second condition, known as negative evidence, measures how well the theoretical pulse explains the empirical peaks in the IOI histogram. It penalizes the candidate period if there are massive peaks in the histogram that do not align with any integer multiple or fraction of the candidate period.[10]

Alternatively, a simplified harmonic support score $S_{harmonic}(c_i)$ can be computed directly by applying a weighting function to related clusters.[3]

$$S_{harmonic}(c_i) = A(c_i) + \sum_{j \neq i} A(c_j) \cdot W(c_i, c_j)$$

In this equation, $A(c)$ represents the amplitude of the peak at centroid c, and $W(c_i, c_j)$ is a piecewise weighting function that returns a maximum value if c_j is a small integer multiple or divisor of c_i, and zero otherwise.[3]

$$ W(c_i, c_j) =

\begin{cases}

1.0 & \text{if } c_j \approx 2 c_i \text{ or } c_j \approx 3 c_i \text{ or } c_j \approx \frac{1}{2} c_i \

0.5 & \text{if } c_j \approx 4 c_i \text{ or } c_j \approx \frac{1}{3} c_i \

0.0 & \text{otherwise}

\end{cases} $$

The candidate c_{best} that minimizes the TWM error function (or maximizes the harmonic support score) is selected as the estimated dominant pulse, representing the inter-beat interval (IBI) in samples. The system then mathematically translates this sample distance into the overarching estimated tempo:

$$\text{BPM} = \frac{60 \cdot F_s}{c_{best}}$$

This mechanism elegantly solves the missing pulse problem. Even if the downbeat is entirely absent throughout the beatboxing performance, the mathematical summation of integer-ratio periodicities from the surrounding syncopated hits forces the silent fundamental subdivision to emerge as the highest-scoring theoretical peak.

3. Multi-Agent Beat Tracking Architecture

While the IOI histogram successfully extracts the global frequency (tempo) of the underlying metrical grid, it possesses two critical limitations. First, it cannot determine the phase of the beat, meaning it does not know which specific transient index marks the start of the musical measure. Second, the histogram represents a static, global average; it cannot adapt to continuous tempo drift, such as an intentional ritardando performed by the beatboxer.[3]

To resolve these limitations and establish a dynamic musical grid, the system must transition from static analysis to a dynamic tracking phase. This is achieved by deploying a multi-agent tracking architecture, fundamentally inspired by Simon Dixon's BeatRoot logic.[3]

3.1. Defining the Mathematical Agent State

In the context of a strict C++ implementation, an "Agent" is a deterministic state machine representing a singular, evolving hypothesis regarding the tempo and phase of the music. Because human timing is inherently ambiguous, a single hypothesis is insufficient. The system must instantiate multiple agents to explore divergent interpretations of the transient array simultaneously, allowing the mathematically optimal path to emerge over time.[3]

An Agent is defined by a continuous state vector containing the following parameters:

- P_n : The current period, representing the expected inter-beat interval in samples.
- ϕ_n : The current phase, representing the exact sample index of the most recently accepted beat.
- S_n : The cumulative reliability score of the agent's hypothesis.
- H : The tracking history, maintaining the sequential chain of accepted beat indices.

The engine initializes the agent pool utilizing the highest-scoring periodicities extracted from the IOI histogram analysis. For each candidate period P_0, the system generates a distinct agent. Furthermore, because the initial phase is unknown, the system clones these agents and assigns their starting phase ϕ_0 to correspond sequentially with the first few transient hits in the array T.[3] This ensures that all plausible starting positions and tempos are actively monitored.

3.2. Agent Prediction, Tolerance Windows, and Branching Logic

As the algorithm iteratively processes the chronologically sorted transient array T, every active agent attempts to predict the exact sample index of the next beat based on its internal state. The prediction is formulated as:

$$\text{Prediction}_{next} = \phi_n + P_n$$

Due to the micro-timing fluctuations inherent to human beatboxing, an incoming transient t_i will almost never align perfectly with the predicted mathematical index. To accommodate this variance, each agent utilizes two mathematically defined tolerance boundaries: an Inner Window (W_{inner}) and an Outer Window (W_{outer}).[3]

Let the base tolerance deviation Δ be defined as a proportional fraction of the agent's current period, typically $\Delta = 0.1 \cdot P_n$. The windows are established relative to the prediction:

- $W_{inner} =$
- $W_{outer} =$

When evaluating an incoming transient t_i against an agent's prediction, the routing logic follows strict deterministic conditions, detailed in the table below.

Condition	Mathematical Truth	Agent Action	Rhythmic Implication
Inner Match	$t_i \in W_{inner}$	Accepts t_i as the true beat. Updates	The transient is a strong, on-beat hit with

		ϕ_n and P_n.	minor micro-timing deviation.
Outer Match	$t_i \in W_{outer} \setminus W_{inner}$	Branches (clones) into two distinct agents.	The transient is highly ambiguous. It may be a rushed/dragged beat, or a syncopated off-beat.
Miss / Rest	$t_i \notin W_{outer}$	Advances phase mathematically without accepting a hit. Applies penalty.	The transient is entirely unrelated to the current beat, implying a missing pulse or rest at the predicted location.

The branching logic triggered by an Outer Window match is the core mechanism that allows the system to recover from tracking errors.[14] When t_i falls in the outer window, the system cannot definitively ascertain if the human beatboxer rushed the beat significantly, or if they played a heavily swung 16th note just prior to the true beat. The agent clones itself: Branch A assumes the transient is the true beat and aggressively updates its state to absorb the massive error. Branch B assumes the transient is merely a syncopated decoration, ignores the hit, maintains its current period, and projects its next prediction to $\phi_n + 2P_n$.[14] Both agents continue tracking, and the mathematical scoring function ultimately decides which hypothesis was correct.

3.3. Phase and Period Correction for Tempo Drift

When an agent formally accepts an incoming transient t_i as a valid beat, it must reconcile the difference between its prediction and the actual acoustic reality. The temporal error is mathematically defined as $e = t_i - (\text{Prediction}_{next})$.

To lock onto the human performance, the agent immediately resets its phase to align perfectly with the transient:

$$\phi_{n+1} = t_i$$

However, the agent cannot instantly replace its internal period with the newly observed interval, as doing so would cause the tempo map to violently fluctuate in response to every minor performance inaccuracy. Instead, the agent updates its period proportionally to the error, effectively acting as a low-pass discrete filter. This allows the agent to ignore high-frequency jitter while successfully tracking low-frequency sustained tempo changes, such as a ritardando.[14]

The period update equation is formulated as:

$$P_{n+1} = P_n + \alpha \cdot e$$

Here, $\alpha \in (0, 1)$ represents the learning rate or tracking responsiveness. A typical α value for processing expressive human timing ranges between 0.15 and 0.25. This mathematical restraint ensures that the agent smoothly adapts to an accelerating beatbox performance without losing its foundational metrical stability.[14] If the agent encounters a missing beat, no transient is accepted. The phase is updated theoretically ($\phi_{n+1} = \text{Prediction}_{next}$), but the period remains strictly unchanged ($P_{n+1} = P_n$), as there is no acoustic evidence to justify a tempo modification.[14]

4. Objective Scoring Functions and Dynamic Programming Formulation

The branching logic utilized by the agents inherently produces an exponential explosion of hypotheses. To prevent systemic collapse and to ultimately identify the correct tempo map, the system relies on a rigorous mathematical scoring function. The survival of an agent is dictated by its cumulative score S_n, which dynamically balances three conflicting parameters: regularity (consistency of the inter-beat period), continuity (minimization of missed beats), and salience (successful matching with actual transients).[17]

4.1. The Transition Cost Penalty

Drawing upon objective functions utilized in Dynamic Programming (DP) beat tracking models, the score update must heavily penalize any agent that stretches its period unnaturally to capture distant, unrelated transients.[4] When an agent accepts a hit t_i, the score is updated

using a function that balances a positive reward for the match against a transition cost penalty.

The transition cost function $F(\Delta t, P_n)$ measures the penalty for deviations from the agent's expected tempo period. A robust mathematical implementation uses a squared-error function applied to the log-ratio of the actual time spacing to the ideal spacing.[18]

$$F(t_i - \phi_n, P_n) = -\left[\log_2\left(\frac{t_i - \phi_n}{P_n}\right)\right]^2$$

This logarithmic penalty function is highly advantageous because it is symmetric on a log-time axis. An interval that is exactly twice as long as the expected period receives the exact same mathematical penalty as an interval that is exactly half as long. The function reaches its absolute maximum value of zero when the actual interval flawlessly matches the expected period.

The agent's cumulative score is updated as follows:

$$S_{n+1} = S_n + O(t_i) + \lambda \cdot F(t_i - \phi_n, P_n)$$

In this formulation, $O(t_i)$ represents a localized reward for matching a transient. In systems utilizing onset envelopes, this can be the exact amplitude of the transient.[18] In an unquantized index array where amplitude data is stripped, $O(t_i)$ is simply a constant reward of 1.0. The coefficient λ is a weighting constant used to balance the local match against the strictness of the tempo consistency.

If an agent's prediction results in a missed beat, its score suffers a severe linear decay:

$$S_{n+1} = S_n - P_{penalty}$$

4.2. Memory and Complexity Management within C++

In a strict C++ environment targeting WebAssembly, the dynamic allocation of objects must be fiercely controlled to prevent heap fragmentation and memory leaks. The branching logic creates a scenario where the number of agents could exceed millions if left unchecked.[14]

The algorithm executes a deterministic culling phase following the evaluation of every transient index.

1. **Hypothesis Merging:** If two distinct agents converge such that their predicted phases and periods are nearly identical ($|\phi_A - \phi_B| < \epsilon$ and $|P_A - P_B| < \epsilon$), their future behaviors will be identical, rendering one mathematically redundant. The agent with the lower cumulative score is immediately terminated and its memory recycled.[14]
2. **Relative Thresholding:** Any agent whose score falls significantly below the current global maximum score across the pool ($S_i < S_{max} - S_{drop}$) is deallocated.
3. **Absolute Capping:** To guarantee hard real-time execution constraints, the entire vector of active agents is sorted by score using an $O(N \log N)$ algorithm, and only the top K agents (e.g., $K = 100$) are permitted to process the next transient.

To further optimize memory, agents do not store their entire beat history array as a dynamic std::vector, which would require massive reallocation during cloning. Instead, the engine utilizes a static memory pool of history nodes. Each agent simply stores a pointer to its current leaf node. When an agent branches, it creates a new node pointing back to its parent, establishing a lightweight, linked tree structure. Once the final transient is processed, the system identifies the singular agent with the absolute highest score and traces its pointers backward to reconstruct the definitive, unquantized metrical grid.[3]

5. Constrained Optimization: Integrating User Anchors

While the probabilistic multi-agent tracker is highly resilient, purely mathematical models can fail when subjected to wildly erratic beatboxing performances where syncopation causes the dominant pulse to vanish for extended durations. To rectify algorithmic misalignment, a user may inject an "Anchor"—a hard mathematical constraint defining an exact, irrefutable metrical alignment.

For example, an anchor A_1 dictates: *The transient at sample index 88200 represents exactly Bar 2, Beat 1.*

Mathematically, an Anchor acts as a boundary condition $A = (t_{anchor}, M_{anchor})$, where t_{anchor} is the precise sample index and M_{anchor} is the target metrical coordinate (e.g., MIDI ticks). The injection of this hard constraint fundamentally intercepts the multi-agent logic, requiring the system to execute both forward state forcing and backward recursive propagation.[4]

5.1. Forward State Forcing and Re-initialization

When the algorithm tracks chronologically forward from the time of the anchor t_{anchor}, the anchor acts as an absolute state collapse for the agent pool. Any agent history accumulated

prior to t_{anchor} that does not perfectly intersect this required coordinate is invalidated.

The algorithm forcefully re-initializes a new pool of agents originating exclusively at t_{anchor}.

- **Absolute Phase Lock:** All newly generated agents are strictly initialized with $\phi_0 = t_{anchor}$.
- **Period Recalculation:** The foundational periods P_0 for these agents are drawn from a re-calculated localized IOI histogram originating from t_{anchor}, ensuring that the forward-tracking tempo adapts exclusively to the rhythmic nature of the new musical section.
- **Score Normalization:** The cumulative tracking scores are normalized to zero, preventing post-anchor tracking paths from being unfairly biased by successful pre-anchor performance.

During standard chronological processing, if an active agent encounters t_{anchor}, a brutal culling heuristic is applied. Any agent whose mathematically predicted beat ϕ_n does not fall within an extremely narrow ϵ-window of t_{anchor} is immediately terminated. The surviving agents are geometrically forced to snap their phase exactly to t_{anchor}, resetting their internal error matrices.

5.2. Bidirectional Propagation and Viterbi Pathfinding

The complexity increases significantly if a user places an anchor deep within the audio file, necessitating the recalculation of the grid *backward* from that specific sample index.[19] Within the C++ architecture, temporal directionality is agnostic. The array of transients preceding the anchor is mathematically reversed, producing a subset $T_{rev} = [t_{anchor}, t_{k-1}, t_{k-2}, \dots, t_1]$.

The identical multi-agent tracking logic described in Section 3 is executed upon T_{rev}.

- The prediction equation is inverted: $\text{Prediction}_{prev} = \phi_n - P_n$.
- Phase and period error updates operate symmetrically, propagating the human tempo drift backward toward the inception of the file.

When a user defines *two* anchors, $A_1 = (t_1, M_1)$ and $A_2 = (t_2, M_2)$, the temporal space between them becomes a rigidly defined mathematical corridor. The total number of

beats B between A_1 and A_2 is known with absolute certainty: $B = M_2 - M_1$.

This boundary configuration transforms the multi-agent hypothesis problem into a constrained Dynamic Programming (DP) or Viterbi optimization problem.[4] The system knows the precise number of metrical steps B required to bridge the gap between t_1 and t_2. The average expected period is strictly defined as $\overline{P} = (t_2 - t_1)/B$.

The engine constructs a discrete DP cost matrix $C[i, j]$ where i represents the beat index (from 0 to B) and j represents the transient index. The transition cost mapping from state $C[i-1, k]$ to state $C[i, j]$ relies on the log-ratio squared-error penalty defined in Section 4.1 against the enforced average period $\overline{P}$.

To enforce the hard anchor constraints, the boundary conditions of the DP matrix are strictly initialized to positive infinity outside of the anchor coordinates:

$$C[0, t_1] = 0, \quad C[0, j \neq t_1] = \infty$$

$$C = 0, \quad C = \infty$$

By forcing the cost function to ∞ outside the specified anchor nodes, the recursive DP algorithm guarantees an optimal path that perfectly intersects both user constraints. The log-ratio penalty mathematically enforces the smooth distribution of temporal error across the intervening transients, effectively expanding or compressing the local tempo mapping seamlessly to satisfy the user's explicit command without generating sudden, jarring tempo spikes.

6. Fractional Rhythmic Grids: Mixed Meters and Tuplets

Once the foundational tempo map (the sequence of main beat indices) is firmly established and anchored, the C++ engine must map this array into fractional, subdivisional musical grids. Users may override a specific section with a distinct Time Signature (e.g., locking a measure to 7/8) or define a localized Tuplet (e.g., forcing 7 transient hits into the space of 4).

Mathematically, these overrides do not destroy or alter the underlying primary tempo map nodes derived by the tracking agents; instead, they alter the piecewise interpolation of the

local grid lines.[5]

6.1. Continuous Mapping of the Metrical Space

To translate discrete audio sample indices into continuous musical time, the algorithm defines a mapping function $f : S \to M$, where S represents the sample index and M represents MIDI ticks. (Standard MIDI resolution is universally defined as 480 Pulses Per Quarter note, or PPQ).

The tempo map output by the multi-agent tracker yields a definitive set of node pairs $N_i = (t_i, m_i)$, where t_i is the sample index of the accepted beat i, and m_i is its integer metrical position (e.g., $i \times 480$).

For any arbitrary sample s that falls between the bounds of t_i and t_{i+1}, the corresponding tick value is calculated via piecewise linear interpolation:

$$m(s) = m_i + (s - t_i)\frac{m_{i+1} - m_i}{t_{i+1} - t_i}$$

The slope of this interpolation function represents the instantaneous tempo between the two beats. By mapping transients in this manner, the system elegantly handles ritardandos, as the physical sample distance $(t_{i+1} - t_i)$ widens, decreasing the slope of the function and lowering the instantaneous BPM.

6.2. Tuplet Calculations and Local Grid Warping

A tuplet $X : Y$ (e.g., $7 : 4$, representing 7 notes played in the duration normally occupied by 4) is mathematically treated as a local scaling of the metrical space.[5] Crucially, the instantiation of a tuplet does not manipulate the primary tempo map nodes N_i; it solely dictates the generation of the interior sub-grid.

If a user designates a $7 : 4$ tuplet beginning at node N_i and spanning exactly 4 quarter notes (terminating at N_{i+4}), the total sample duration of this physical span is determined by the tempo map: $D_{samples} = t_{i+4} - t_i$.

The total metrical duration is $D_{ticks} = 4 \times 480 = 1920$ ticks.

Under standard division, an 8th note commands 240 ticks. Under the constraints of the $7:4$ tuplet, the span must be partitioned into 7 equal metrical segments. The new local subdivision tick length is computed as:

$$\text{Tick}_{tuplet} = \frac{D_{ticks}}{7} \approx 274.285 \text{ ticks}$$

To mathematically generate the precise sample indices for snapping unquantized transient hits inside this tuplet, the C++ engine iterates over the sample domain:

$$\text{Grid}_k = t_i + k \cdot \frac{D_{samples}}{7}, \quad \text{for } k \in \{0, 1, \dots, 7\}$$

Any raw transient hit point residing within the domain $[t_i, t_{i+4}]$ is subsequently quantized to the nearest Grid_k. Because this specific calculation relies rigidly on the outer boundaries t_i and t_{i+4}, the overarching multi-agent tempo map remains perfectly continuous and intact. The tuplet operates solely as a rendering-layer mathematical transformation.[21]

Rhythmic Element	**Mathematical Operation**	**Effect on Tempo Map (Ni)**
Standard Grid	$m(s) = m_i + \Delta s \cdot \text{slope}$	None. Follows established node pairs.
Tuplet Division	$\text{Grid}_k = t_i + k \cdot (D_{samples}$	None. Internal piecewise scaling only.
Time Sig (Additive)	$\text{Bar} = m_i + (\text{Beats} \times 480$	None. Moves theoretical barline coordinate.
Metric Modulation	$P_{new} = P_{old} \times (Y_{old}/Y_{new}$	Modifies agent tracking period dynamically.

6.3. Mixed Meters and Metric Modulation

A Time Signature override (e.g., shifting the meter from $4/4$ to $7/8$) mathematically redefines the expected distance between measure boundaries and, depending on the composer's intent, potentially triggers a complex metric modulation.[6]

When the denominator of the time signature fluctuates, the C++ engine must determine the specific mathematical pivot value.[22] The system accounts for two distinct mathematical scenarios:

Scenario A: Constant Quarter Note (Additive Meter Override)

If the tempo remains constant but the signature shifts to $7/8$, the physical length of the measure simply decreases. If a $4/4$ measure spanned 4 beats ($4 \times 480 = 1920$ ticks), the subsequent $7/8$ measure spans exactly 3.5 beats ($3.5 \times 480 = 1680$ ticks). In this scenario, the multi-agent tracker remains entirely unaffected; the agents continue tracking standard quarter notes. The engine simply inserts a barline structural marker at the $m = 1680$ tick mark rather than the 1920 tick mark.

Scenario B: Constant Eighth Note (Metric Modulation) If the beat subdivision remains constant but the macroscopic beat grouping changes (e.g., transitioning from $4/4$ to $6/8$ where the dotted-quarter note becomes the new foundational beat), the underlying tempo undergoes a mathematical modulation.[6]

Let the preceding tempo be BPM_{old}, where the quarter note represents 1 beat. In the new $6/8$ meter, the dotted-quarter note represents 1 beat. Because $1 \text{ Dotted Quarter} = 1.5 \text{ Quarter Notes}$, the new mapped tempo period P_{new} required by the tracking agents must be adjusted instantaneously:

$$P_{new} = P_{old} \times 1.5$$

$$BPM_{new} = BPM_{old} \times \frac{1}{1.5}$$

If a user injects this Time Signature override at a specific transient $t_{override}$, the C++ engine

immediately intercepts the active agent pool and scales the current period P_n of all surviving agents by the fractional equivalent derived from the respective time signatures.

This mathematical scaling guarantees that the agents seamlessly widen or narrow their inner and outer tolerance windows exactly at the downbeat of the new meter, successfully capturing the human performer's intentional metric modulation without causing the tracking agents to fail and without discarding the highly valuable historical tracking data.

Through the rigorous application of these deterministic, mathematical methodologies—ranging from all-order IOI histogram smoothing to DP-constrained multi-agent phase locking—the C++ engine effectively maps the chaotic nature of human beatboxing into a structured, highly accurate, and manipulatable digital musical grid.

Works cited

1. Pulse Detection in Syncopated Rhythms using Neural Oscillators - Music Dynamics Laboratory, accessed April 23, 2026, https://musicdynamicslab.uconn.edu/wp-content/uploads/sites/433/2016/03/VelascoLarge2011ISMIRPubsAHEdits.pdf
2. Neural Entrainment to the Beat: The “Missing-Pulse” Phenomenon - PMC - NIH, accessed April 23, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC5490067/
3. A Lightweight Multi-Agent Musical Beat Tracking System - Austrian Research Institute for Artificial Intelligence, accessed April 23, 2026, https://ofai.at/papers/oefai-tr-2000-08.pdf
4. Beat Tracking by Dynamic Programming - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/publication/249816846_Beat_Tracking_by_Dynamic_Programming
5. Time signature - Wikipedia, accessed April 23, 2026, https://en.wikipedia.org/wiki/Time_signature
6. The math of tempo and signature - Cubase - Steinberg Forums, accessed April 23, 2026, https://forums.steinberg.net/t/the-math-of-tempo-and-signature/614289
7. (PDF) Evaluation of Audio Beat Tracking and Music Tempo Extraction Algorithms - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/publication/248906423_Evaluation_of_Audio_Beat_Tracking_and_Music_Tempo_Extraction_Algorithms
8. estimation of tempo, micro time and time signature from percussive music - International Conference on Digital Audio Effects (DAFx), accessed April 23, 2026, https://www.dafx.de/paper-archive/2003/pdfs/dafx46.pdf
9. A Real-time Beat Tracking System for Audio Signals, accessed April 23, 2026, https://staff.aist.go.jp/m.goto/PAPER/ICMAS96goto.pdf
10. A COMPUTATIONAL APPROACH TO RHYTHM DESCRIPTION AUDIO FEATURES FOR THE COMPUTATION OF RHYTHM PERIODICITY FUNCTIONS AND THEIR USE, accessed April 23, 2026,

http://www.mtg.upf.edu/files/publications/9d0455-PhD-Gouyon.pdf
11. A COMPUTATIONAL APPROACH TO RHYTHM DESCRIPTION AUDIO FEATURES FOR THE COMPUTATION OF RHYTHM PERIODICITY FUNCTIONS AND THEIR USE, accessed April 23, 2026, https://www.tdx.cat/bitstream/handle/10803/7484/tfg1de1.pdf?sequence=1
12. Generation of Musical Scores of Percussive Un-Pitched Instruments from Automatically Detected Events, accessed April 23, 2026, https://www.iis.fraunhofer.de/content/dam/iis/de/doc/ame/conference/AES-116-Convention_GeneratingMusicalScoresofPercussiveInstruments_AES6032.pdf
13. Time Signature Detection: A Survey - MDPI, accessed April 23, 2026, https://www.mdpi.com/1424-8220/21/19/6494
14. Automatic Extraction of Tempo and Beat from Expressive Performances - Austrian Research Institute for Artificial Intelligence, accessed April 23, 2026, https://ofai.at/papers/oefai-tr-2001-19.pdf
15. Beat Tracking for Multiple Applications: A Multi-Agent System Architecture With State Recovery - SciSpace, accessed April 23, 2026, https://scispace.com/pdf/beat-tracking-for-multiple-applications-a-multi-agent-system-498g2bhehz.pdf
16. Analysis of Musical Content in Digital Audio - Austrian Research Institute for Artificial Intelligence, accessed April 23, 2026, https://ofai.at/papers/oefai-tr-2002-39.pdf
17. An Interactive Beat Tracking and Visualisation System - ResearchGate, accessed April 23, 2026, https://www.researchgate.net/publication/2382205_An_Interactive_Beat_Tracking_and_Visualisation_System
18. Beat Tracking by Dynamic Programming - Columbia University, accessed April 23, 2026, https://www.ee.columbia.edu/~dpwe/pubs/Ellis07-beattrack.pdf
19. A TWO-FOLD DYNAMIC PROGRAMMING APPROACH TO BEAT TRACKING FOR AUDIO MUSIC WITH TIME-VARYING TEMPO - ISMIR 2011, accessed April 23, 2026, https://ismir2011.ismir.net/papers/PS2-4.pdf
20. Tuplets in Music: Exploring Complex Rhythmic Divisions | by Myk Eff | Sound & Design, accessed April 23, 2026, https://soundand.design/tuplets-in-music-exploring-complex-rhythmic-divisions-29ae923f9cc2
21. Let's Talk Rhythm Part 2: Nested Tuplets - KLANG, accessed April 23, 2026, http://klangnewmusic.weebly.com/direct-sound/lets-talk-rhythm-part-2-nested-tuplets
22. Chapter 23: Metric Modulation – The Rhythm and Meter Compendium, accessed April 23, 2026, https://openbooks.library.baylor.edu/rhythm/chapter/23/
23. Mixing Meters, Triplets, and Syncopation - Rhythm 4 - YouTube, accessed April 23, 2026, https://www.youtube.com/watch?v=TsXo0ifmHQY